Homework Assignment 1

Anaisía Brown

2026-10-07

```
#shortcut for installing necessary packages.
knitr::opts_chunk$set(echo = TRUE)
knitr::opts_chunk$set(warning=FALSE, message = FALSE)

packages <- c("tidygeocoder", "sf", "terra", "tidyverse", "tidycensus", "flextable", "tmap")
installed <- packages %in% rownames(installed.packages())
if (any(!installed)) {install.packages(packages[!installed])}
lapply(packages, library, character.only = TRUE)
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
## terra 1.9.50
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x tidyr::extract() masks terra::extract()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
##
## Attaching package: 'flextable'
##
##
## The following object is masked from 'package:purrr':
##
##     compose
##
##
## The following objects are masked from 'package:terra':
##
##     align, colorize, rotate, width
##
##
## Registered S3 method overwritten by 'stars':
##   method                  from
##   st_interpolate_aw.stars sf
```

```
## [[1]]
## [1] "tidygeocoder" "stats"        "graphics"     "grDevices"    "utils"
## [6] "datasets"     "methods"      "base"
##
## [[2]]
## [1] "sf"           "tidygeocoder" "stats"        "graphics"     "grDevices"
## [6] "utils"        "datasets"     "methods"      "base"
##
## [[3]]
##  [1] "terra"        "sf"           "tidygeocoder" "stats"        "graphics"
##  [6] "grDevices"    "utils"        "datasets"     "methods"      "base"
##
## [[4]]
##  [1] "lubridate"    "forcats"      "stringr"      "dplyr"        "purrr"
##  [6] "readr"        "tidyr"        "tibble"       "ggplot2"      "tidyverse"
## [11] "terra"        "sf"           "tidygeocoder" "stats"        "graphics"
## [16] "grDevices"    "utils"        "datasets"     "methods"      "base"
##
## [[5]]
##  [1] "tidycensus"   "lubridate"    "forcats"      "stringr"      "dplyr"
##  [6] "purrr"        "readr"        "tidyr"        "tibble"       "ggplot2"
## [11] "tidyverse"    "terra"        "sf"           "tidygeocoder" "stats"
## [16] "graphics"     "grDevices"    "utils"        "datasets"     "methods"
## [21] "base"
##
## [[6]]
##  [1] "flextable"    "tidycensus"   "lubridate"    "forcats"      "stringr"
##  [6] "dplyr"        "purrr"        "readr"        "tidyr"        "tibble"
## [11] "ggplot2"      "tidyverse"    "terra"        "sf"           "tidygeocoder"
## [16] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [21] "methods"      "base"
##
## [[7]]
##  [1] "tmap"         "flextable"    "tidycensus"   "lubridate"    "forcats"
##  [6] "stringr"      "dplyr"        "purrr"        "readr"        "tidyr"
## [11] "tibble"       "ggplot2"      "tidyverse"    "terra"        "sf"
## [16] "tidygeocoder" "stats"        "graphics"     "grDevices"    "utils"
## [21] "datasets"     "methods"      "base"
```

##Geocoding

#Q1. Import this In N Out Burger CSV file.

```
#part a.
download.file("https://raw.githubusercontent.com/pjames-ucdavis/SPH215/refs/heads/main/innout_final.csv

#part b.
in_n_out <- read_csv("innout_final.csv")
```

#Q2. Glimpse at the data. What is the address field that you might want to geocode?

```
We would want to geocode the "store address" field.
```

```
#Glimpse at data.
glimpse(in_n_out)
```

```
## Rows: 30
## Columns: 2
## $ `Store Number`  <dbl> 131, 327, 276, 217, 225, 247, 320, 319, 197, 207, 143,~
## $ `store address` <chr> "3501 Truxel Road Sacramento CA 95834", "8200 Delta Sh~
```

```
#To view data as an Excel document.
view(in_n_out)

#Look specifically at "store address" column.
head(in_n_out$`store address`)
```

```
## [1] "3501 Truxel Road Sacramento CA 95834"
## [2] "8200 Delta Shores Circle Sacramento CA 95832"
## [3] "2001 Alta Arden Expy. Sacramento CA 95825"
## [4] "4600 Madison Ave. Sacramento CA 95841"
## [5] "780 Ikea Court West Sacramento CA 95605"
## [6] "2900 Del Paso Rd. Sacramento CA 95834"
```

#Q3. Geocode the data using the "arcgis" geocoder in the tidygeocoder package.

```
in_n_out_geo <- geocode(in_n_out,"store address", method = "arcgis")
head(in_n_out_geo)
```

```
## # A tibble: 6 x 4
##   `Store Number` `store address`                                  lat  long
##            <dbl> <chr>                                          <dbl> <dbl>
## 1            131 3501 Truxel Road Sacramento CA 95834            38.6 -122.
## 2            327 8200 Delta Shores Circle Sacramento CA 95832    38.5 -121.
## 3            276 2001 Alta Arden Expy. Sacramento CA 95825       38.6 -121.
## 4            217 4600 Madison Ave. Sacramento CA 95841           38.7 -121.
## 5            225 780 Ikea Court West Sacramento CA 95605         38.6 -122.
## 6            247 2900 Del Paso Rd. Sacramento CA 95834           38.7 -122.
```

#Q4. Take a look at the lat and long columns. Do we have missing data? Based on the summary reports, there are no missing (NAs) addresses for the latitude or the longitude.

```
#check for missing data in each column

#latitude
summary(in_n_out_geo$lat)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   35.99   37.99   38.39   38.12   38.59   38.76
```

```
#longitude
summary(in_n_out_geo$long)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -122.7  -122.0  -121.5  -121.5  -121.3  -119.7
```

#Q5.Make the data set spatial with a CRS of WGS 84 (hint: crs=4326).

```
#Specify the lat, long coordinates for R
in_n_out_points <- st_as_sf(in_n_out_geo, coords = c("long", "lat"))

#check if there's a CRS.
st_crs(in_n_out_points)           #ResultCRS:NA
```

```
## Coordinate Reference System: NA
```

```
#Set CRS of WGS 84
in_n_out_crs <- st_set_crs(in_n_out_points, 4326)

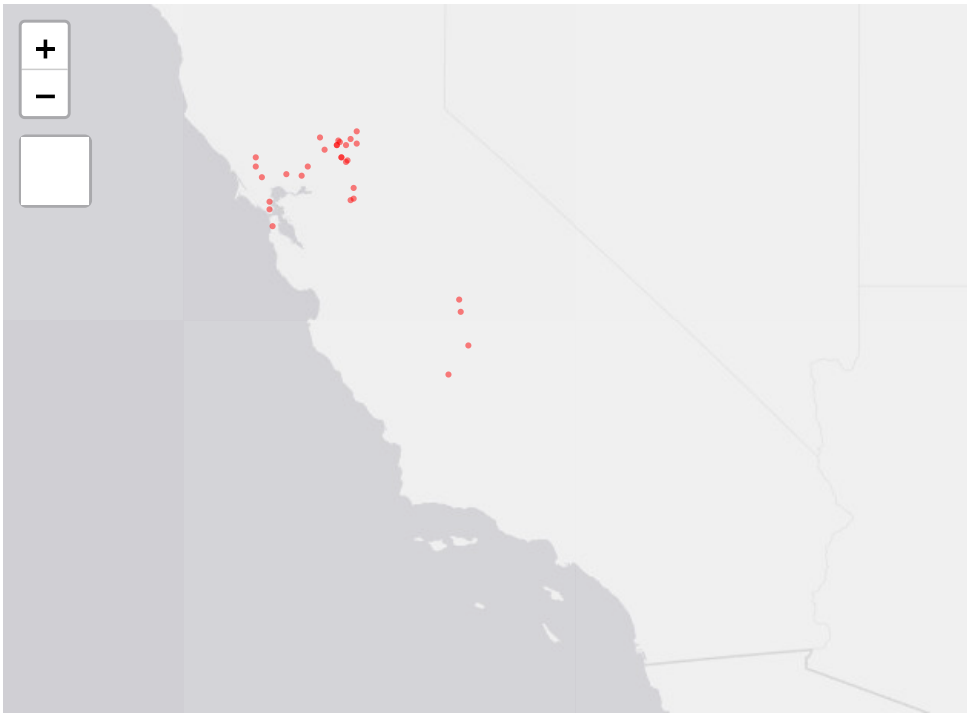
#Check the CRS
st_crs(in_n_out_crs)
```

```
## Coordinate Reference System:
##   User input: EPSG:4326
##   wkt:
## GEOGCRS["WGS 84",
##     ENSEMBLE["World Geodetic System 1984 ensemble",
##         MEMBER["World Geodetic System 1984 (Transit)"],
##         MEMBER["World Geodetic System 1984 (G730)"],
##         MEMBER["World Geodetic System 1984 (G873)"],
##         MEMBER["World Geodetic System 1984 (G1150)"],
##         MEMBER["World Geodetic System 1984 (G1674)"],
##         MEMBER["World Geodetic System 1984 (G1762)"],
##         MEMBER["World Geodetic System 1984 (G2139)"],
##         MEMBER["World Geodetic System 1984 (G2296)"],
##         ELLIPSOID["WGS 84",6378137,298.257223563,
##             LENGTHUNIT["metre",1]],
##         ENSEMBLEACCURACY[2.0]],
##     PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##         AXIS["geodetic latitude (Lat)",north,
##             ORDER[1],
##             ANGLEUNIT["degree",0.0174532925199433]],
##         AXIS["geodetic longitude (Lon)",east,
##             ORDER[2],
##             ANGLEUNIT["degree",0.0174532925199433]],
##     USAGE[
##         SCOPE["Horizontal component of 3D system."],
##         AREA["World."],
##         BBOX[-90,-180,90,180]],
##     ID["EPSG",4326]]
```

#Q6. Map the addresses using the tmap package. Make the points red.

```r
#Establish Map View
tmap_mode("view")

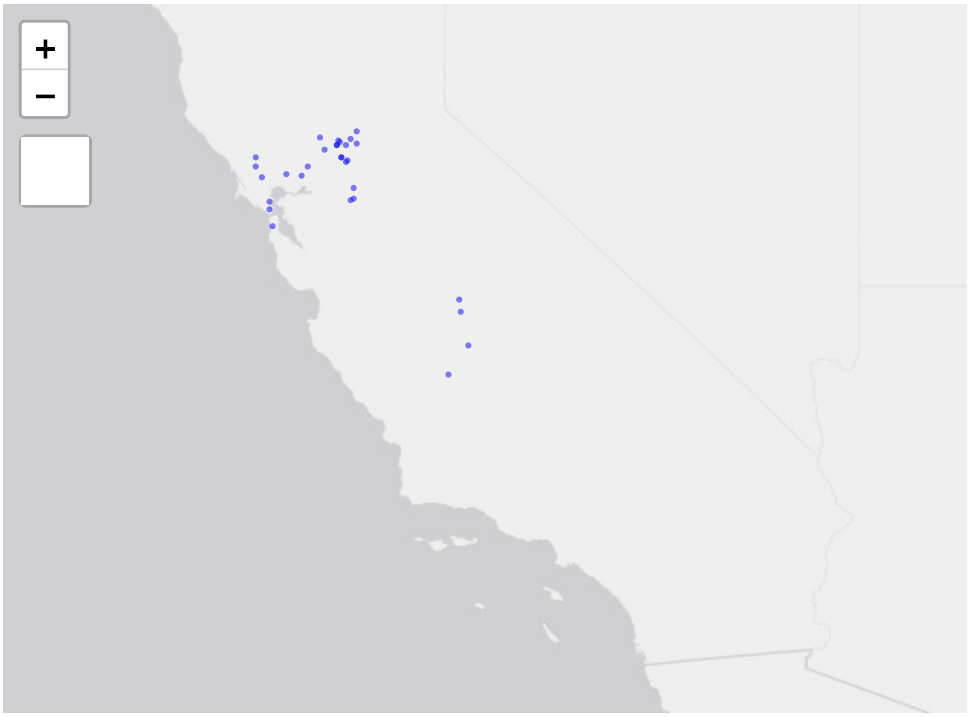
#Map w/ Red Points
burger_map = tm_shape(in_n_out_crs) + tm_dots( fill = "red", size= 0.3, fill_alpha = 0.5)
burger_map
```



#Q7. Make blue points!

```r
#Establish Map View
tmap_mode("view")

#Map w/ Blue Points
burger_map_b = tm_shape(in_n_out_crs) + tm_dots( fill = "blue", size= 0.3, fill_alpha = 0.5)
burger_map_b
```



##Map Point Health Data

#Q8. Import the CAdata dataset from the MapGAM package. The MapGAM package wouldn't load, so I

imported the data from the class GitHUb.

```
#CAdata Dataset
download.file("https://raw.githubusercontent.com/pjames-ucdavis/SPH215/refs/heads/main/CAdata.rds", "CA
```

```
#read CAdata
cancer <- readRDS("CAdata.rds")
```

```
#View dataset
summary(cancer)
```

```
##       time               event              X                 Y
##  Min.   : 0.004068   Min.   :0.0000   Min.   :1811375   Min.   :-241999
##  1st Qu.: 1.931247   1st Qu.:0.0000   1st Qu.:2018363   1st Qu.: -94700
##  Median : 4.749980   Median :1.0000   Median :2325084   Median : -60386
##  Mean   : 6.496130   Mean   :0.6062   Mean   :2230219   Mean   :  87591
##  3rd Qu.: 9.609031   3rd Qu.:1.0000   3rd Qu.:2380230   3rd Qu.: 318280
##  Max.   :24.997764   Max.   :1.0000   Max.   :2705633   Max.   : 770658
##       AGE         INS
##  Min.   :25.00   Mcd: 431
##  1st Qu.:53.00   Mcr:1419
##  Median :62.00   Mng:2304
##  Mean   :61.28   Oth: 526
##  3rd Qu.:71.00   Uni: 168
##  Max.   :80.00   Unk: 152
```

```
view (cancer)
```

#Q9. Add a coordinate reference system to the CAdata dataset and make sure it's a spatial dataset that you can map.

```
#Define the "X" and "Y" coordinates for R.
cancer_points <- st_as_sf(cancer, coords=c("X","Y"))

#Identify if there's a CRS.
st_crs(cancer_points)          #CRSResult: NA
```

```
## Coordinate Reference System: NA
```

```
#Load CRS dataset
ca_projection <- "+proj=lcc +lat_1=40 +lat_2=41.66666666666666
             +lat_0=39.33333333333334 +lon_0=-122 +x_0=2000000
             +y_0=500000.0000000002 +ellps=GRS80
             +datum=NAD83 +units=m +no_defs"
#Add CRS
cancer_points_crs <- st_set_crs(cancer_points, ca_projection)

#Check CRS
st_crs(cancer_points_crs)
```

```
## Coordinate Reference System:
##   User input: +proj=lcc +lat_1=40 +lat_2=41.66666666666666
```

```
##              +lat_0=39.33333333333334 +lon_0=-122 +x_0=2000000
##              +y_0=500000.0000000002 +ellps=GRS80
##              +datum=NAD83 +units=m +no_defs
##   wkt:
## PROJCRS["unknown",
##     BASEGEOGCRS["unknown",
##         DATUM["North American Datum 1983",
##             ELLIPSOID["GRS 1980",6378137,298.257222101,
##                 LENGTHUNIT["metre",1]],
##             ID["EPSG",6269]],
##         PRIMEM["Greenwich",0,
##             ANGLEUNIT["degree",0.0174532925199433],
##             ID["EPSG",8901]]],
##     CONVERSION["unknown",
##         METHOD["Lambert Conic Conformal (2SP)",
##             ID["EPSG",9802]],
##         PARAMETER["Latitude of false origin",39.3333333333333,
##             ANGLEUNIT["degree",0.0174532925199433],
##             ID["EPSG",8821]],
##         PARAMETER["Longitude of false origin",-122,
##             ANGLEUNIT["degree",0.0174532925199433],
##             ID["EPSG",8822]],
##         PARAMETER["Latitude of 1st standard parallel",40,
##             ANGLEUNIT["degree",0.0174532925199433],
##             ID["EPSG",8823]],
##         PARAMETER["Latitude of 2nd standard parallel",41.6666666666667,
##             ANGLEUNIT["degree",0.0174532925199433],
##             ID["EPSG",8824]],
##         PARAMETER["Easting at false origin",2000000,
##             LENGTHUNIT["metre",1],
##             ID["EPSG",8826]],
##         PARAMETER["Northing at false origin",500000,
##             LENGTHUNIT["metre",1],
##             ID["EPSG",8827]]],
##     CS[Cartesian,2],
##         AXIS["(E)",east,
##             ORDER[1],
##             LENGTHUNIT["metre",1,
##                 ID["EPSG",9001]]],
##         AXIS["(N)",north,
##             ORDER[2],
##             LENGTHUNIT["metre",1,
##                 ID["EPSG",9001]]]]
```

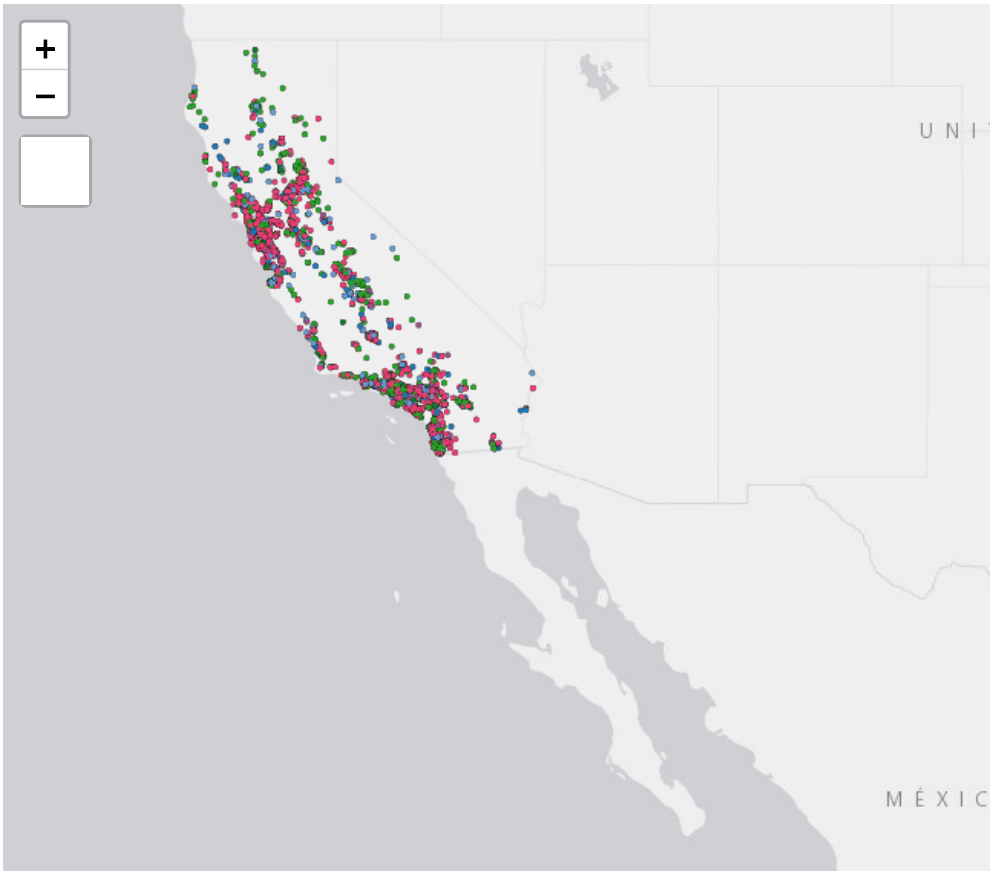
```
#Confirmed it's a spatial data set!
```

#Q10. Create a map of the CAdata data set with points color coded by Insurance Status using the tmap package.

```
#Establish Map Interactive Mode
tmap_mode("view")

#Construct a map based on "Insurance Status in CA"
insur_map = tm_shape(cancer_points_crs) + tm_dots(fill="INS", fill.scale=tm_scale_categorical())
```

```
#View Map
insur_map
```



##Census Data

#Q11. Use the tidycensus package to download American Community Survey Five year estimates from

2018-2022 in California at the Census tract level for the following variable: a. Household Median Income = "B19013_001"

```
#census_api_key("86fccf503d859dae6884ddca2dbf66c8372bc5c7", install = TRUE)"
#API Key saved to R Environment - comment out to avoid multiple downloads

#Loading variable in ACS data set from 2018-2022
acs2022 <- load_variables(2022, "acs5", cache = TRUE)
view(acs2022)

#Extract Household Median Income data from 2018-2022
ca <- get_acs(geography = "tract",
              year = 2022,
              variables = c(medincome = "B19013_001"),
              state = "CA",
              survey = "acs5",
              output = "wide")
ca
```

```
## # A tibble: 9,129 x 4
##    GEOID       NAME                                           medincomeE medincomeM
##    <chr>       <chr>                                               <dbl>      <dbl>
##  1 06001400100 Census Tract 4001; Alameda County; Califor~     234236      42845
##  2 06001400200 Census Tract 4002; Alameda County; Califor~     225500      29169
##  3 06001400300 Census Tract 4003; Alameda County; Califor~     164000      44675
##  4 06001400400 Census Tract 4004; Alameda County; Califor~     158836       9038
##  5 06001400500 Census Tract 4005; Alameda County; Califor~      95078      34125
##  6 06001400600 Census Tract 4006; Alameda County; Califor~     155893      33824
##  7 06001400700 Census Tract 4007; Alameda County; Califor~     110556      12318
##  8 06001400800 Census Tract 4008; Alameda County; Califor~      97132      37490
##  9 06001400900 Census Tract 4009; Alameda County; Califor~     114583      31849
## 10 06001401000 Census Tract 4010; Alameda County; Califor~     153654      31601
## # i 9,119 more rows
```

#Q12. Take a glimpse at your data

```
glimpse(ca)
```

```
## Rows: 9,129
## Columns: 4
## $ GEOID      <chr> "06001400100", "06001400200", "06001400300", "06001400400",~
## $ NAME       <chr> "Census Tract 4001; Alameda County; California", "Census Tr~
## $ medincomeE <dbl> 234236, 225500, 164000, 158836, 95078, 155893, 110556, 9713~
## $ medincomeM <dbl> 42845, 29169, 44675, 9038, 34125, 33824, 12318, 37490, 3184~
```

#Q13. Give us the mean median household income across tracts in California.

```
#Descriptive statistics Include na.rm = TRUE argument to omit "missingness".

ca %>%
summarize(Mean = mean(medincomeE, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##     Mean
##    <dbl>
## 1 99732.
```

```
#The mean median household income across tracts in CA is $99,732.02.
```

#Q14. Make a table showing the mean, median, and standard deviation of median household income across tract in California, and make it look fancy!

```
#The mean, median, and SD of median household incomes acorss tract in CA
tract <- ca %>%
summarize (Mean = mean(medincomeE, na.rm = TRUE),
   Median = median(medincomeE, na.rm = TRUE),
   SD = sd(medincomeE, na.rm = TRUE))

#Table summary of Measure of Central Tendency
ttable  <- flextable (tract)
ttable
```

Mean	Median	SD
99,732.02	90,230	46,278.62

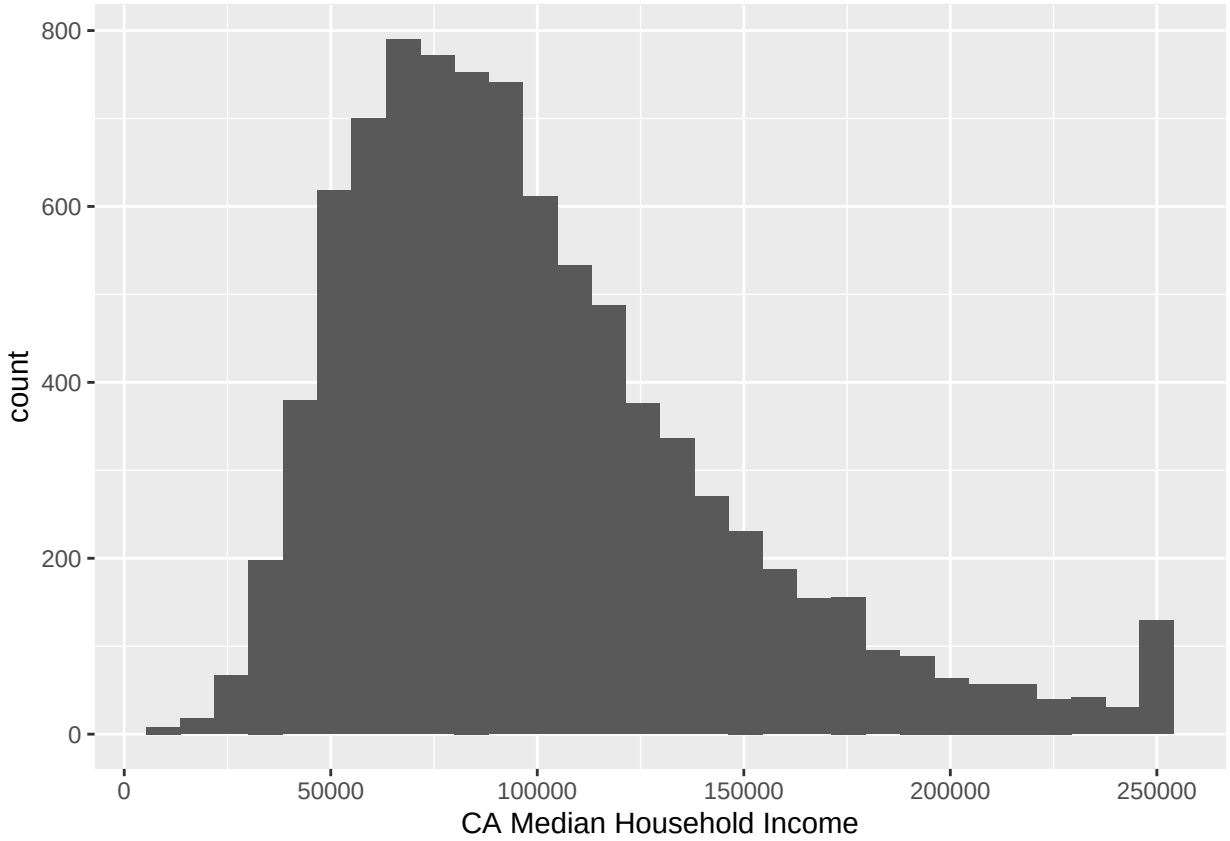
```
#Table w/ clearer labels
tract_table <- ttable %>%
  set_header_labels(Mean ="Mean", Median = "Median", SD = "Standard Deviation") %>% flextable::align(al

tract_table
```

Mean	Median	Standard Deviation
99,732.02	90,230	46,278.62

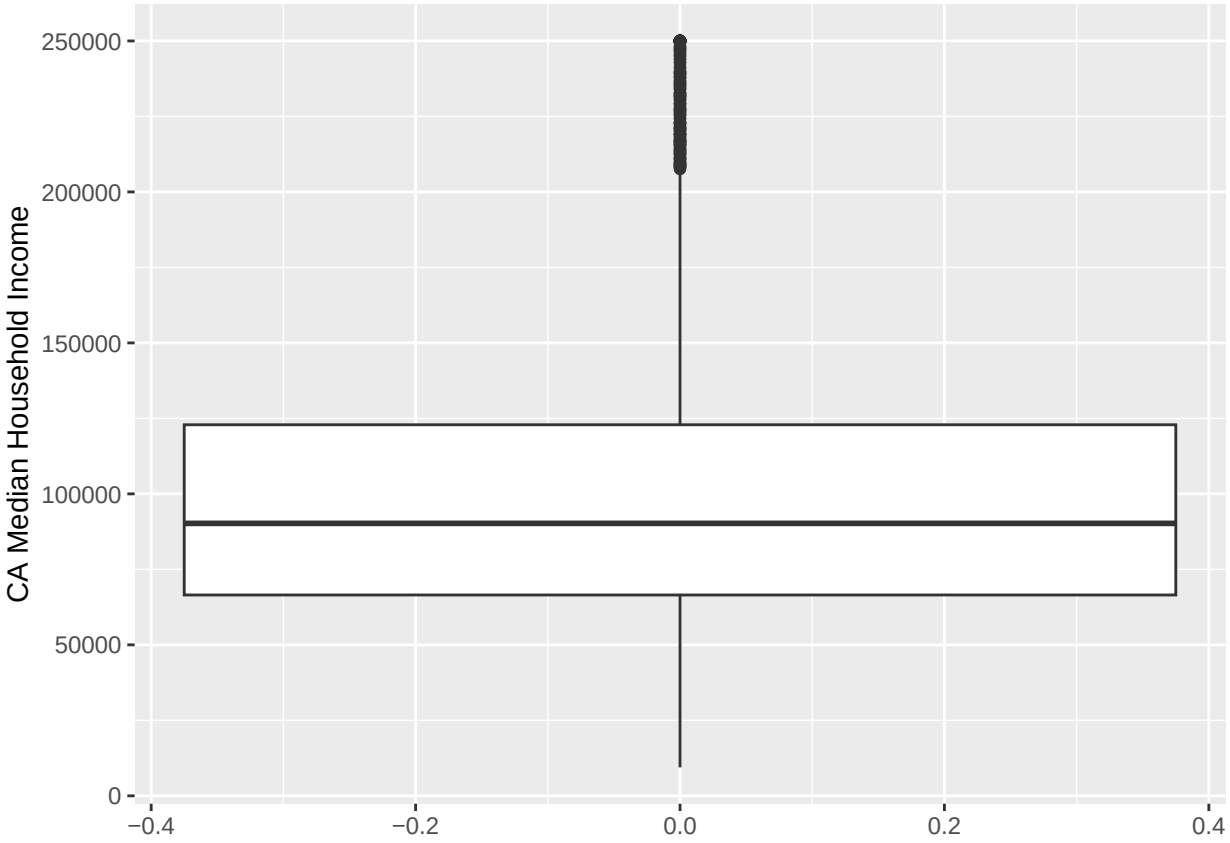
#Q15. Create a histogram of median household income across tracts in California. Make sure it has a label.

```
#create histogram across tracts in CA
ca %>%
  ggplot() +
  geom_histogram(mapping = aes(x=medincomeE)) + xlab("CA Median Household Income")
```



#Q16. OK now give me a boxplot! And I want a nice label for this too.

```
#create boxplot for MHI across CA tract
ca %>%
  ggplot() +
  geom_boxplot(mapping = aes(y = medincomeE,)) + ylab("CA Median Household Income")
```


CA Median Household Income
250000
200000
150000
100000
50000
0
−0.4
−0.2
0.0
0.2
0.4